

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Fakulta riadenia a informatiky

Dokumentácia k semestrálnemu projektu z predmetu Algoritmy a údajové štruktúry 2

Vypracoval: Ľubomír Bojda

Akademický rok: 2025/2026

Obsah

1 Implementácia AVL stromu ... 3
1.1 Uzol AVL stromu ... 3
1.2 Operácia vloženia ... 3
1.2.1 Metóda RebalanceAfterInsert ... 3
1.3 Operácia mazania ... 6
1.3.1 Metóda RebalanceAfterDelete ... 6
1.4 Operácia hľadania ... 7
2 Porovnanie rýchlosti operácií na údajových štruktúrach ... 7
2.1 Vkladanie ... 8
2.2 Mazanie ... 9
2.3 Vyhľadávanie ... 9
2.4 Intervalové vyhľadávanie ... 10
2.5 Vyhľadanie minimálneho kľúča ... 11
2.6 Vyhľadanie maximálneho kľúča ... 12
3 Systém PCR testov ... 13
3.1 Návrh systému ... 13
3.2 Triedy s dátami ... 13
3.3 Obaľovacie triedy ... 14
3.3.1 Obaľovacie triedy pre osoby ... 14
3.3.2 Obaľovacie triedy pre PCR testy ... 14
3.4 Architektúra systému ... 15
3.5 Zložitosť operácií ... 18
4 Diagram tried ... 22

1 IMPLEMENTÁCIA AVL STROMU

Táto sekcia popisuje implementáciu AVL stromu, realizovanú bez použitia rekurzie, s dôrazom na časovú efektívnosť.

1.1 Uzol AVL stromu

Uzol AVL stomu (AVLTNode<T>) okrem atribútov z uzlu binárneho vyhľadávacieho stromu (ľavý syn, pravý syn, rodič) obsahuje aj atribút, ktorý uchováva informáciu o výške uzla (výška predstavuje dĺžku najdlhšej cesty k listu). Na základe výšky je následne možné vypočítať vyvažovací faktor – počíta sa ako rozdiel výšky ľavého syna a pravého syna. Pre zachovanie vyváženia v AVL strome musí platiť, že vyvažovací faktor každého uzla je z množiny {-1, 0, 1}.

1.2 Operácia vloženia

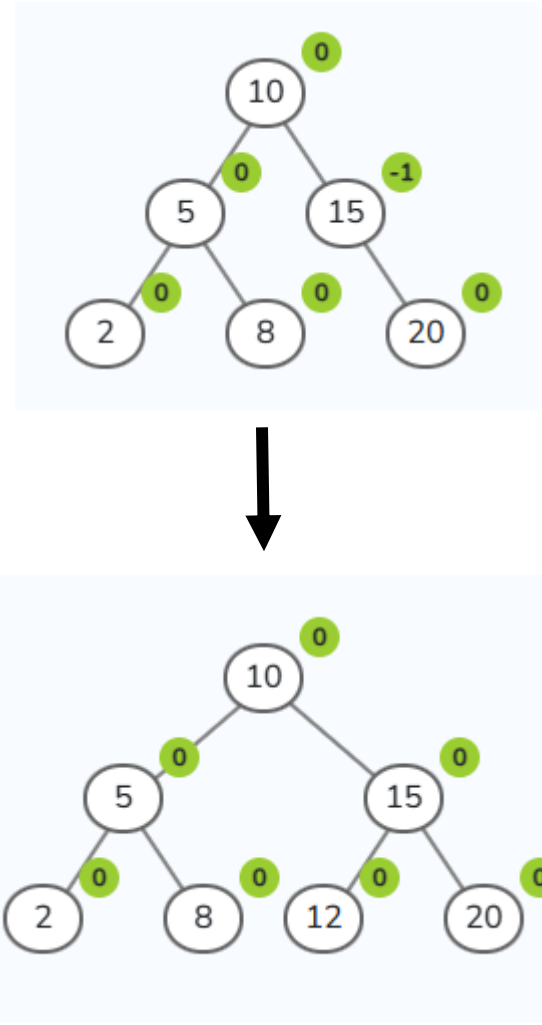
Pomocou protected metódy InsertNode(), ktorú implementuje už BVS sa nájde vhodné miesto a nový uzol sa na toto miesto vloží. Metóda vráti novo vložený uzol, alebo null pokiaľ sa snažíme vkladať uzol sa takým kľúčom, ktorý už v strome existuje. Pre zachovanie vlastnosti vyváženia AVL stromu je následne potrebné kontrolovať vyvažovacie faktory uzlov od rodiča novo vloženého uzlu po koreň a vykonávať rotácie na základe hodnoty faktora aktuálne skúmaného uzla. Toto kontrolovanie je možné ukončiť skôr, teda nemusíme skontrolovať uzly až po koreň. Všetko to zabezpečuje metóda RebalanceAfterInsert() popísaná v ďalšej podkapitole.

1.2.1 Metóda RebalanceAfterInsert

Táto metóda vykonáva nasledovné kroky:

- **Prepočítanie výšky uzla:** aktualizuje sa výška aktuálne kontrolovaného uzla.
- **Výpočet vyvažovacieho faktora:** vypočíta sa vyvažovací faktor aktuálne kontrolovaného uzla pomocou výšok jeho synov.
- **Porovnávanie vyvažovacieho faktora (VF):** porovnávame 3 situácie:
    1. VF = 0 – strom je už vyvážený a nie je potrebné pokračovať v kontrolovaní ďalších uzlov.

Vizuálna ukážka:

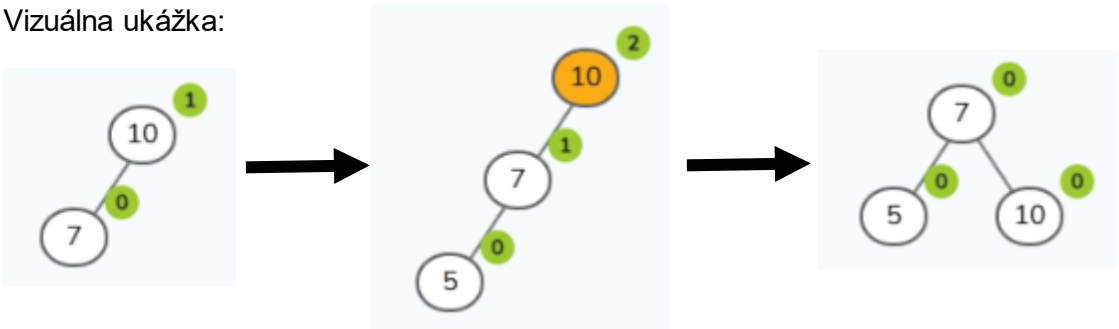


Teda po vložení uzlu s kľúčom 12 by sme začali kontrolovať na uzle 15, pričom jeho VF je po novom 0, takže môžeme vkladanie ukončiť a nepokračovať v kontrole vyššie.

2. VF > 1 – ľavý podstrom je príliš vysoký – tu sa zase rozhoduje z dvoch situácií:

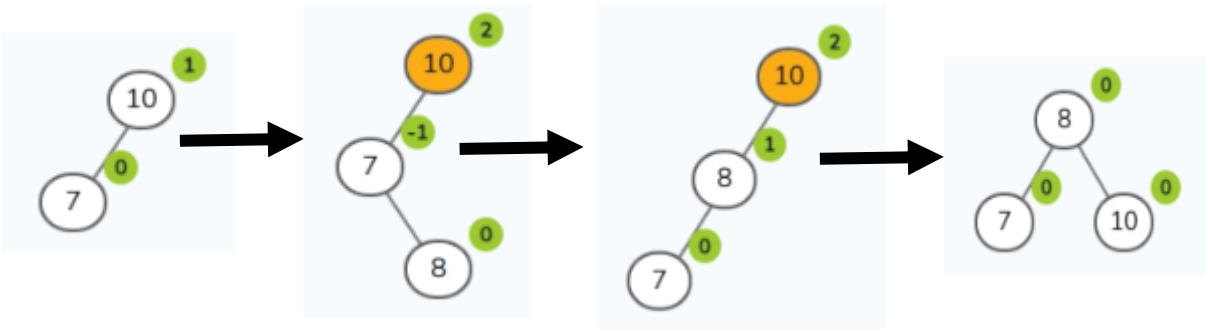
   a) VF ľavého syna >= 0 – vykoná sa iba pravá rotácia na aktuálne kontrolovanom uzle a proces vkladania končí

Vizuálna ukážka:



b) VF ľavého syna < 0 – vykoná sa ľavá rotácia na ľavom synovi aktuálne kontrolovaného uzla a následne pravá rotácia na kontrolovanom uzle a proces vkladania končí

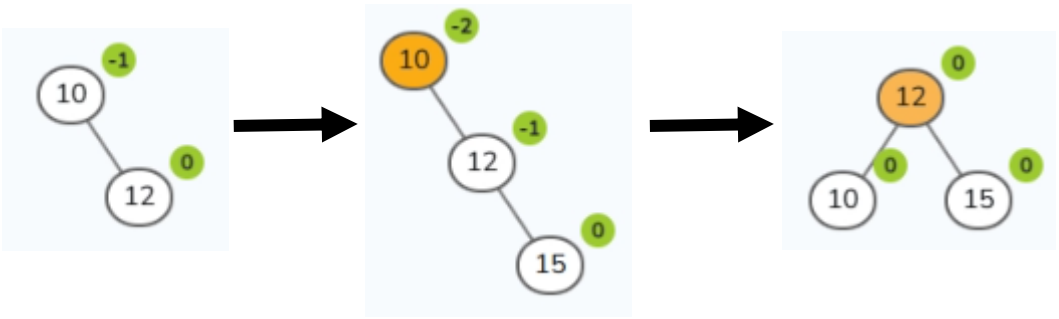
Vizuálna ukážka:



3. VF < -1 - pravý podstrom je príliš vysoký – tu sa zase rozhoduje z dvoch situácií:

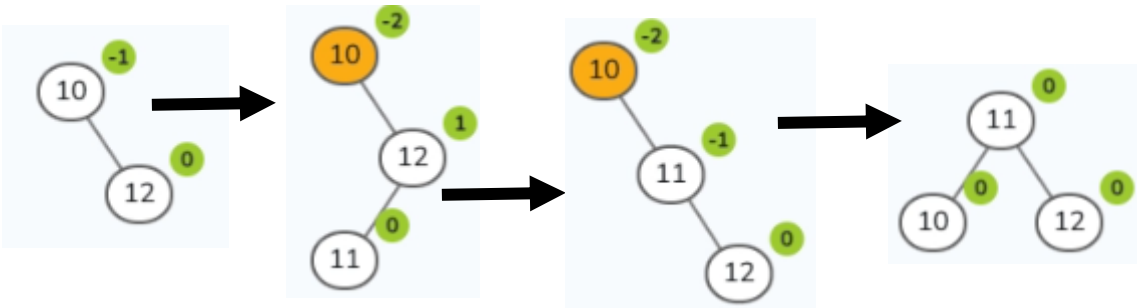
a) VF pravého syna <= 0 - vykoná sa iba ľavá rotácia na aktuálne kontrolovanom uzle a proces vkladania končí

Vizuálna ukážka:



b) VF pravého syna > 0 - vykoná sa pravá rotácia na pravom synovi aktuálne kontrolovaného uzla a následne ľavá rotácia na kontrolovanom uzle a proces vkladania končí

Vizuálna ukážka:



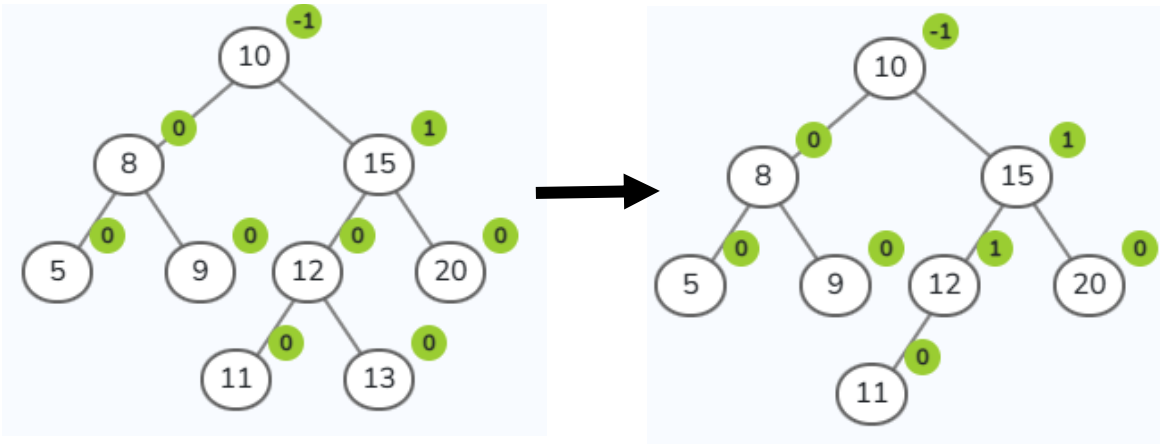
1.3 Operácia mazania

Pri mazaní sa najskôr vykonáva metóda DeleteNode() z BVS, ktorá nájde uzol, ktorý chceme vymazať, vymaže ho a vráti uzol, od ktorého je potrebné kontrolovať vyvažovací faktor smerom ku koreňu pre zachovanie vyváženia AVL stromu. Tento vracaný uzol sa líši od situácie – ak vymazávame uzol, ktorý je list alebo má jedného syna, tak vracaný uzol je rodič mazaného uzla, no ak vymazávame uzol, ktorý má dvoch synov, tak vymeníme mazaný uzol s uzlom z pravého podstromu, ktorý má najmenší kľúč a následne ho môžeme vymazať – vtedy je vracaný uzol rodič uzla z pravého podstromu, s ktorým sme mazaný uzol vymenili. Vymazanie uzla môže spôsobiť kaskádu nerovnováh, ktorá sa vedia ku koreňu, takže nie je možné ukončiť proces mazania hneď po vykonaní jednej rotácie (resp. dvojice rotácií) ako pri vkladaní. Všetko zabezpečuje metóda RebalanceAfterDelete(), ktorá je popísaná v ďalšej podkapitole.

1.3.1 Metóda RebalanceAfterDelete

Táto metóda vykonáva nasledovné kroky:

- **Uloženie starej výšky:** do premennej sa uloží starý výška uzla, teda ešte pred jej prepočítaním.
- **Prepočítanie výšky uzla:** aktualizuje sa výška aktuálne kontrolovaného uzla.
- **Výpočet vyvažovacieho faktora:** vypočíta sa vyvažovací faktor aktuálne kontrolovaného uzla pomocou výšok jeho synov.
- **Porovnávanie vyvažovacieho faktora (VF):** porovnávame vyvažovacie faktory, pričom ošetrujeme 2 situácie, ktoré sú rovnaké, ako situácie číslo 2, 3 pri vkladaní uzla, no s tým rozdielom, že po vykonaní roácie(resp. dvojice rotácií) sa proces mazania nekončí, ale pokračuje ďalej ku koreňu.
- **Kontrola ukončenia:** proces zmeny vyvažovacích faktorov je možné ukončiť, keď nastane nasledovná situácia – nový vyvažovací faktor uzla je ±1 a jeho stará výška je rovnaká ako jeho nová výška. Vizuálna ukážka:



Pred vymazaním uzla 13 mal uzol 12 výšku 1. Po vymazaní ostala jeho výška rovnaká a jeho vyvažovací faktor je +1, teda proces mazania môžeme ukončiť. Tým, že výška uzla 12 ostala rovnaká je zabezpečené, že vyšším uzlom zostáva vyvažovací faktor rovnaký ako predtým.

1.4 Operácia hľadania

Operácia hľadania je úplne rovnaká ako pri BVS, teda nie je potrebná žiadna nová implementácia.

2 POROVNANIE RÝCHLOSTI OPERÁCIÍ NA ÚDAJOVÝCH ŠTRUKTÚRACH

Cieľom tejto kapitoly je porovnať rýchlosť základných operácií nad vybranými stromovými údajovými štruktúrami:

- Binárny vyhľadávací strom,
- AVL strom,
- SortedSet.

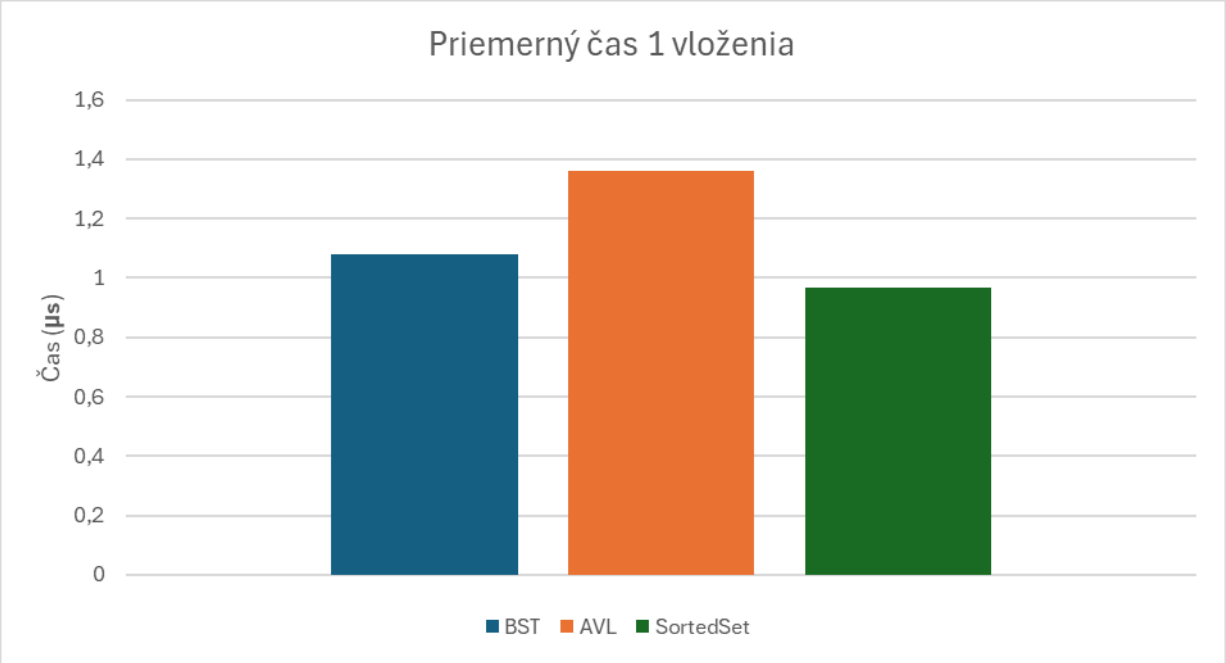
Binárny vyhľadávací strom a AVL strom boli mnou implementované v jazyku C# ako súčasť semestrálnej práce. Aby bolo možné výsledky testov rýchlosti objektívne vyhodnotiť, bola do porovnania zaradená aj trieda SortedSet<T>. Táto trieda predstavuje optimalizovanú implementáciu usporiadanej množiny priamo v štandardnej knižnici .NET. Interné fungovanie SortedSet<T> je založené na Red-Black Tree (červeno-čiernom strome), ktorý patrí medzi samovyvažovacie binárne stromy a zaručuje, že výška stromu zostáva logaritmická vzhľadom na počet prvkov. Vďaka tomu majú operácie ako vkladanie, mazanie a vyhľadávanie garantovanú zložitosť O(log n). Okrem toho poskytuje SortedSet<T> aj pokročilé možnosti, ako napríklad

GetViewBetween(min, max), ktorá umožňuje efektívne získavať všetky prvky nachádzajúce sa v danom intervale hodnôt bez potreby prechádzať celý strom.

2.1 Vkladanie

Do každej štruktúry bolo postupne vložených 10 000 000 náhodne vygenerovaných celých čísel, pričom všetky kľúče boli unikátne, aby sa predišlo duplicite prvkov.

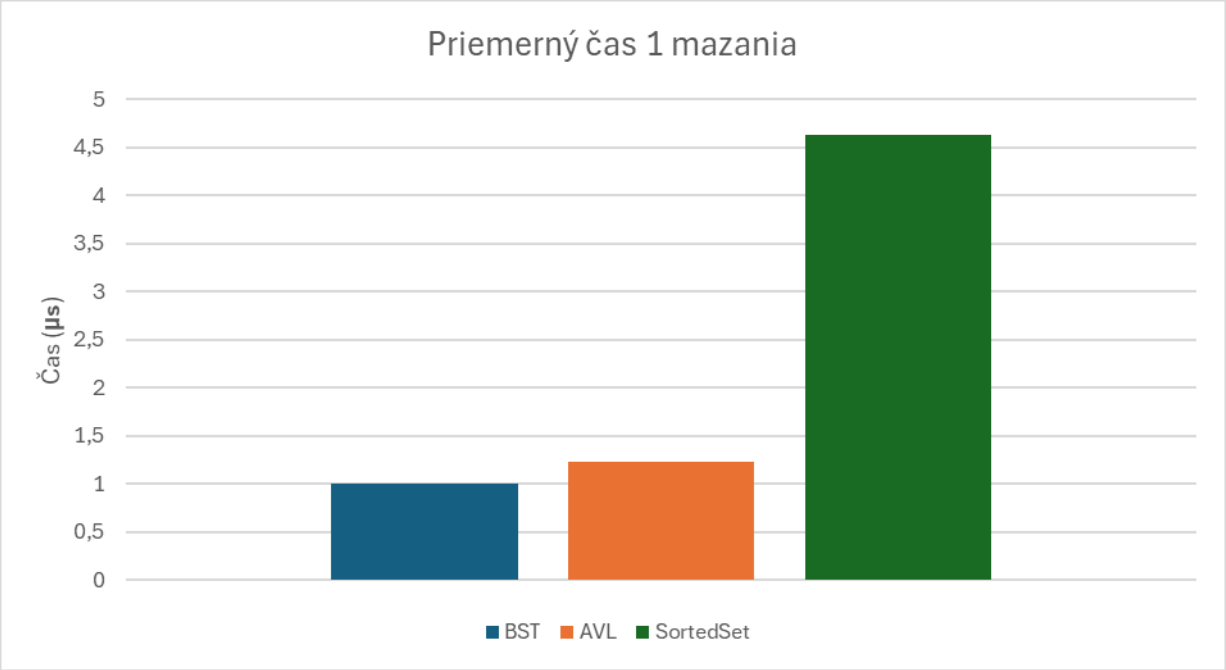
Štruktúra	Celkový čas (ms)	Priemerný čas 1 vloženia (µs)
Binárny vyhľadávací strom	10814,79	1,0814
AVL strom	13597,94	1,3598
SortedSet	9689,26	0,9689



2.2 Mazanie

Z každej štruktúry bolo následne odstránených 2 000 000 prvkov, vybraných náhodne z existujúcich dát. Cieľom testu bolo zmerať, ako efektívne jednotlivé štruktúry zvládajú operáciu mazania a ako sa pri nej prejavuje ich vnútorné vyvažovanie.

Štruktúra	Celkový čas (ms)	Priemerný čas 1 mazania (µs)
Binárny vyhľadávací strom	2005,89	1,0029
AVL strom	2463,02	1,2315
SortedSet	9274,05	4,6370

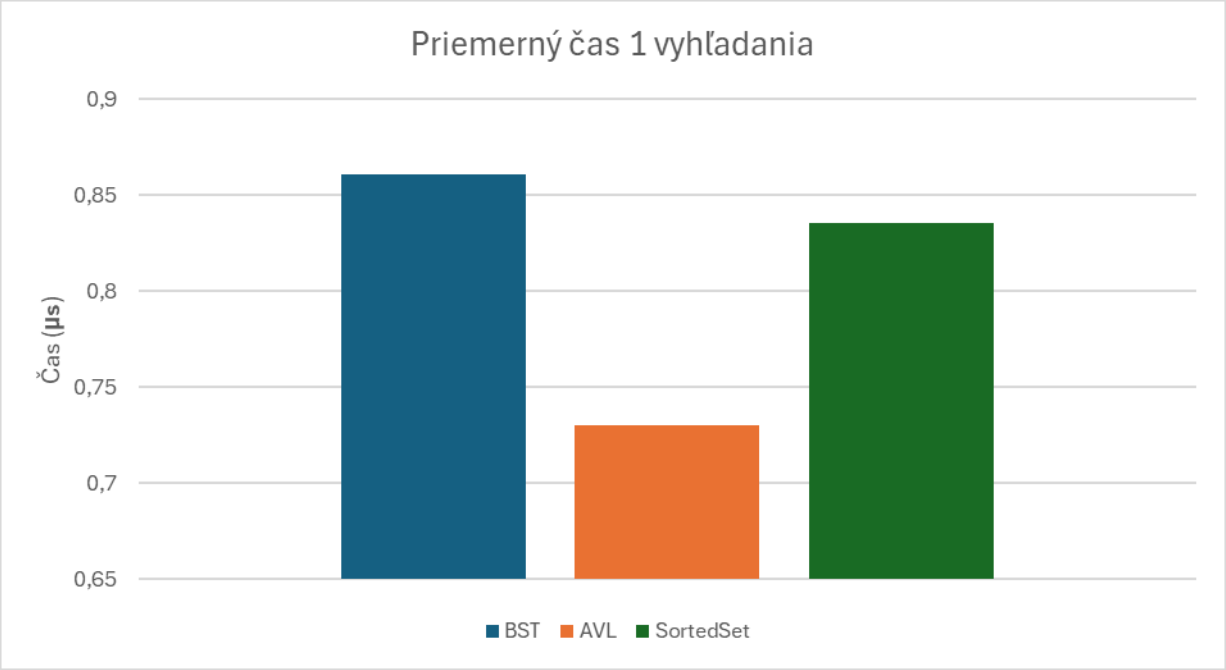


2.3 Vyhľadávanie

V tejto časti bol hodnotený výkon jednotlivých štruktúr pri náhodnom vyhľadávaní existujúcich prvkov. Do testu bolo vybraných 5 000 000 náhodných prvkov, ktoré sa nachádzali v štruktúre.

Štruktúra	Celkový čas (ms)	Priemerný čas 1 vyhľadania (µs)

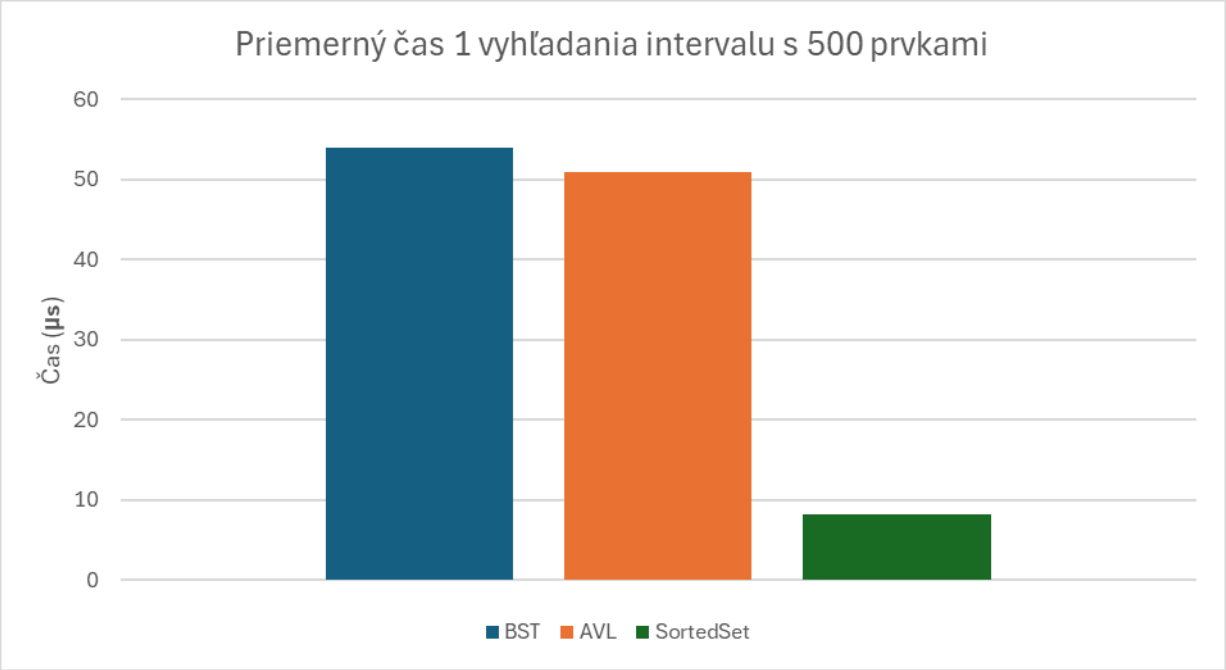
Binárny vyhľadávací strom	4305,77	0,8611
AVL strom	3650,04	0,73
SortedSet	4176,61	0,8353



2.4 Intervalové vyhľadávanie

V tejto časti bol hodnotený výkon jednotlivých štruktúr pri vyhľadávaní všetkých prvkov ležiacich v danom intervale hodnôt. Test bol navrhnutý tak, aby sa v každej operácii našlo 500 prvkov. Intervalové vyhľadávanie bolo opakované 1 000 000 krát, pričom pre všetky tri štruktúry bol použitý rovnaký interval v každej iterácii.

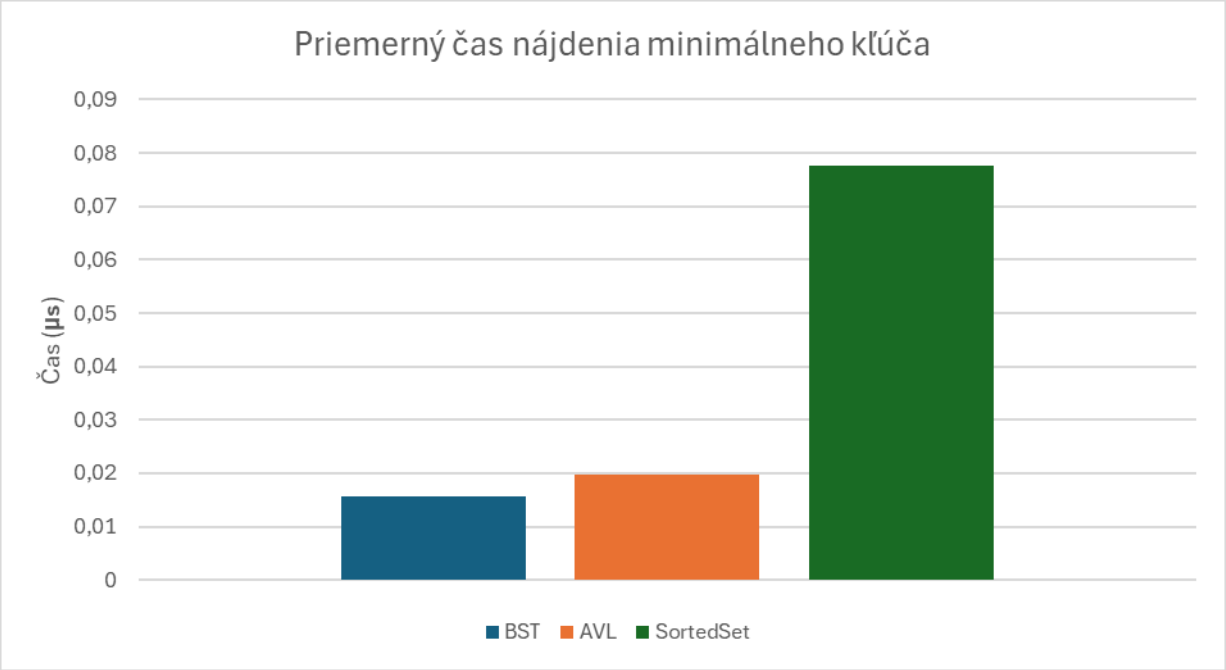
Štruktúra	**Celkový čas (ms)**	**Priemerný čas 1 vyhľadania intervalu s 500 prvkami (µs)**
Binárny vyhľadávací strom	53906,83	53,9068
AVL strom	50950,74	50,9507
SortedSet	8118,95	8,1189



2.5 Vyhľadanie minimálneho kľúča

V tejto časti bol testovaný výkon operácie nájdenia minimálneho kľúča v jednotlivých štruktúrach. Operácia bola opakovaná 2 000 000 krát, aby sa dosiahla stabilná a merateľná hodnota celkového času.

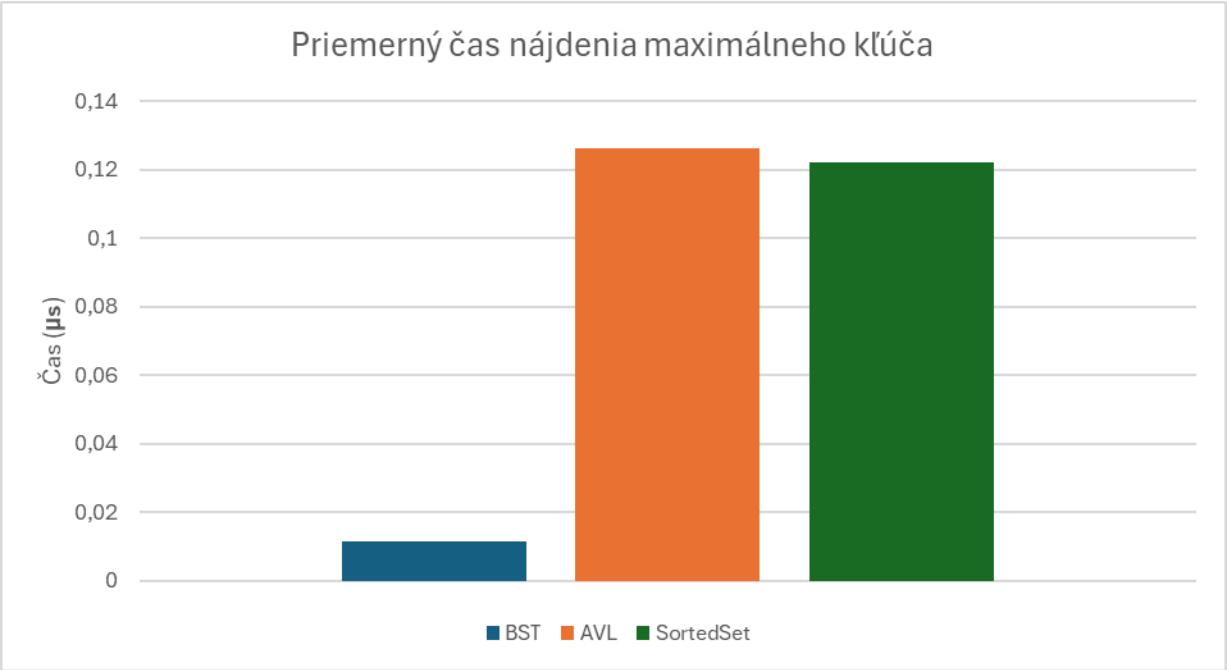
Štruktúra	**Celkový čas (ms)**	**Priemerný čas nájdenia minimálneho kľúča (µs)**
Binárny vyhľadávací strom	31,33	0,0156
AVL strom	39,76	0,0198
SortedSet	155,44	0,0777



2.6 Vyhľadanie maximálneho kľúča

V tejto časti bol testovaný výkon operácie nájdenia maximálneho kľúča v jednotlivých štruktúrach. Operácia bola opakovaná 2 000 000-krát, aby sa dosiahla stabilná a merateľná hodnota celkového času.

Štruktúra	**Celkový čas (ms)**	**Priemerný čas nájdenia maximálneho kľúča (µs)**
Binárny vyhľadávací strom	22,82	0,0114
AVL strom	252,22	0,1261
SortedSet	244,22	0,1221



3 SYSTÉM PCR TESTOV

3.1 Návrh systému

Cieľom návrhu systému PCR testov bola maximálna časová efektivita všetkých 21 požadovaných operácií. Keďže sa celá databáza nachádza v operačnej pamäti a požiadavky na operácie sú rôznorodé (vyhľadávanie podľa ID, podľa času, podľa okresu, podľa kraja, atď.), bolo nutné navrhnúť systém, kde bude zložitosť každej operácie čo najmenšia. Na dosiahnutie tohto cieľa bola zvolená architektúra AVL stromov, ktorá je viacúrovňová (teda uzol stromu môže obsahovať ďalší strom).

3.2 Triedy s dátami

Systém pracuje s dvoma základnými dátovými triedami:

1. Person – uchováva základné údaje o osobe (meno, priezvisko, id, ...).
2. PCRTest – uchováva všetky údaje súvisiace s PCR testom (dátum a čas, id osoby, id testu, id okresu, ...).

Tieto triedy slúžia len ako kontajnery dát, no neobsahujú logiku porovnávania.

3.3 Obaľovacie triedy

Keďže osoby a testy je potrebné triediť podľa rôznych kľúčov (podľa id, kraja, dátumu atď.), tak boli implementované tzv. obaľovacie triedy, ktoré implementujú rozhranie IComparable<>, obsahujú logiku porovnávania pre konkrétny kľúč a uchovávajú referencie na dátové objekty.

3.3.1 Obaľovacie triedy pre osoby

- **PersonByID** – porovnáva osoby na základe ich unikátneho identifikačného čísla,
    - obsahuje dva vnorené stromy – s uzlami typu PCRTestBylD (obsahuje testy usporiadané podľa ich id pre každú osobu) a PCRTestByDateAndTime (obsahuje testy danej osoby usporiadané podľa času vykonania)

3.3.2 Obaľovacie triedy pre PCR testy

- **PCRTestByID** – porovnáva testy na základe ich unikátneho identifikačného čísla.
- **PCRTestByDateAndTime** – porovnáva testy na základe času ich vykonania (čas nemusí byť unikátny, takže ak majú dva testy rovnaký čas vykonania, testy sa porovnajú na základe ich identifikačného čísla).
- **PCRTestByValueReverse** – porovnáva testy na základe ich hodnoty výsledku, ale pri porovnávaní sú hodnoty prehodené, takže vo finále je uzol s najväčším kľúčom prvý pri InOrder prehliadke (takisto nemusí byť unikátna hodnota pre každý test, takže ak sú hodnoty rovnaké, testy sa porovnávajú na základe ich identifikačného čísla).
- **PCRTestByDistrict** – porovnáva testy na základe okresu, v ktorom boli vykonané,
    - obsahuje dva vnorené AVL stromy – s uzlami typov PCRTestByDateAndTime (obsahuje testy usporiadané podľa času v danom okrese) a PCRTestByValueReversed (obsahuje testy usporiadané podľa hodnoty testu reverzne (od najväčšieho po najmenší)).
- **PCRTestByRegion** – porovnáva testy na základe kraja, v ktorom boli vykonané,

- obsahuje vnorený strom – s uzlami typu PCRTestByDateAndTime (obsahuje testy usporiadané podľa času v danom kraji)

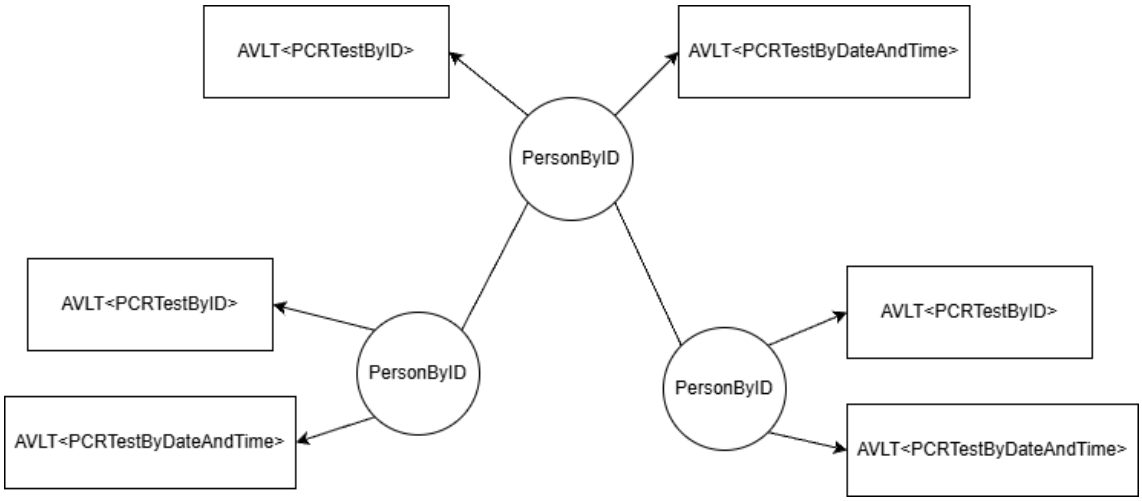
- **PCRTestByTestStation** – porovnáva testy na základe testovacej stanice, v ktorej boli vykonané,
  - obsahuje vnorený strom – s uzlami typu PCRTestByDateAndTime (obsahuje testy usporiadané podľa času v stanici)

Pre triedy PCRTestByDistrict, PCRTestByRegion a PCRTestByTestStation platí, že ich porovnávacia hodnota nemusí byť rovnaká pre každý test, ale tu to nevadí, pretože budem pomocou nich vytvárať len stromy, ktoré budú obsahovať toľko uzlov, koľko je daných hodnôt, teda napríklad pre každý okres bude v strome len jeden uzol a ďalej sa vyhľadáva hodnota v ďalšej úrovni stromov.

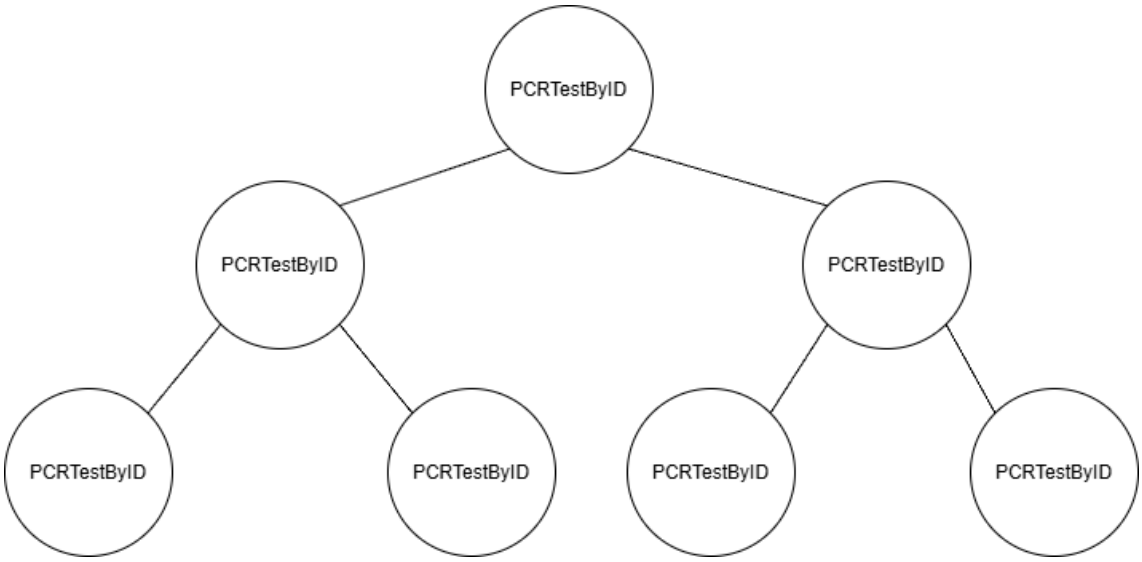
3.4 Architektúra systému

Systém obsahuje 6 hlavných AVL stromov:

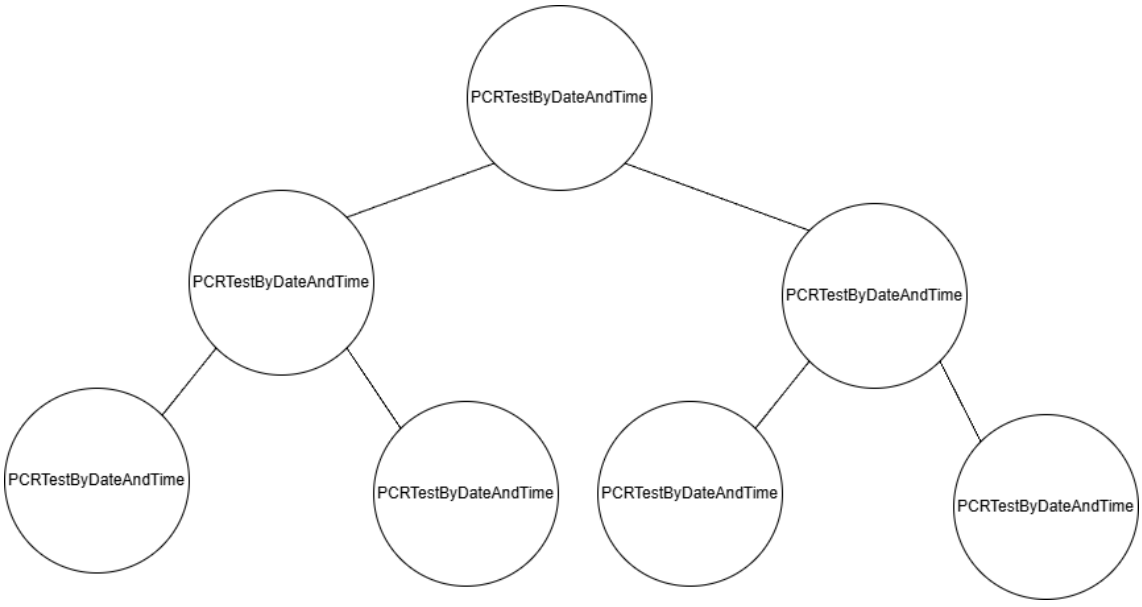
- **AVLT<PersonByID>** - strom, ktorý obsahuje všetky osoby v systéme usporiadané podľa ich identifikačného čísla. Každá osoba v strome obsahuje vlastné stromy AVLT<PCRTestByID> (obsahuje všetky testy pre danú osobu usporiadané podľa id testu) a AVLT<PCRTestByDateAndTime> (obsahuje testy pre danú osobu usporiadané podľa dátumu a času vykonania).



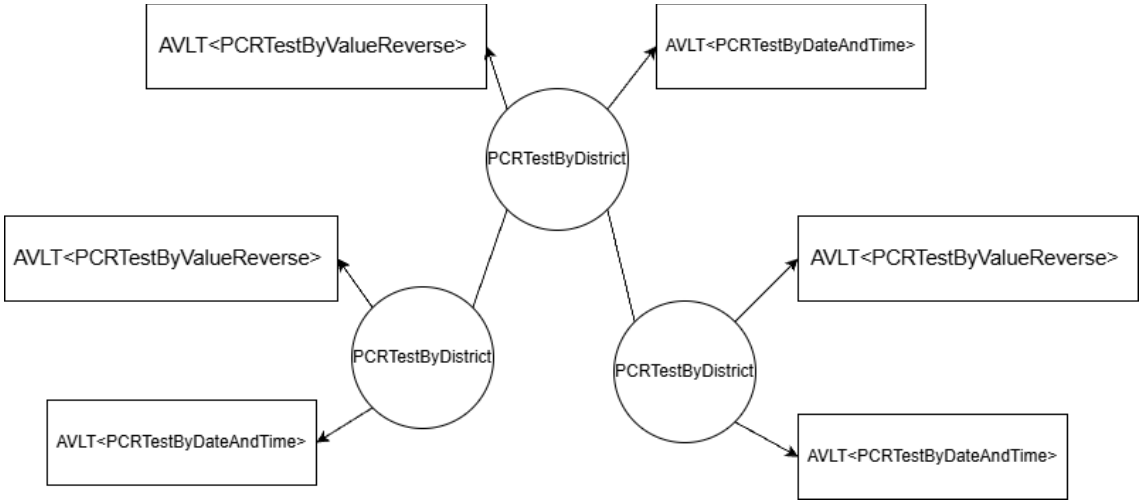
- **AVLT<PCRTestByID> -** strom obsahuje všetky testy v systéme usporiadané podľa ich identifikačného čísla.



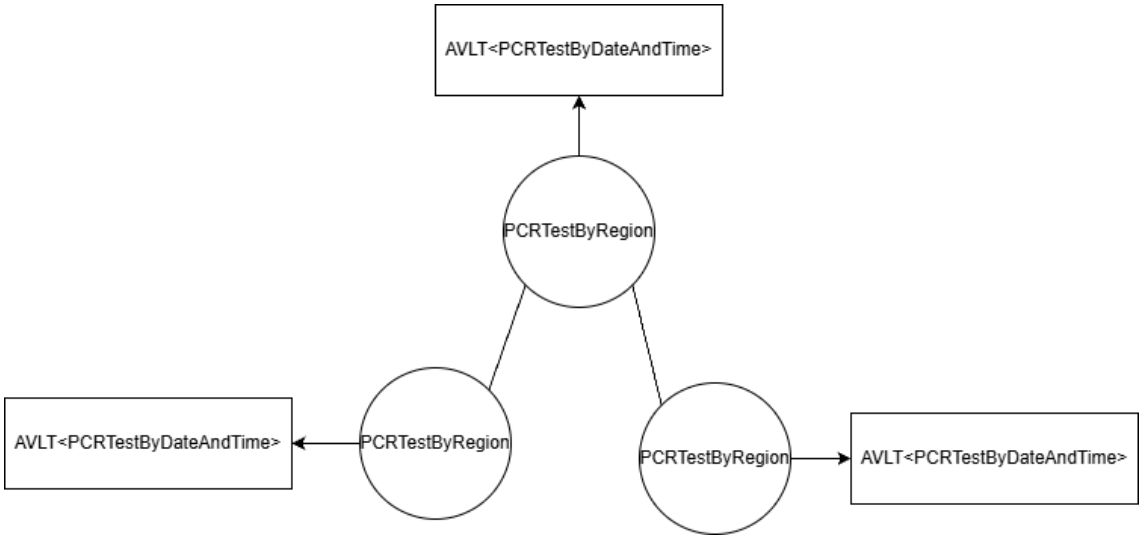
- **AVLT<PCRTestByDateAndTime> -** strom obsahuje všetky testy v systéme usporiadané podľa dátumu a času vykonania.



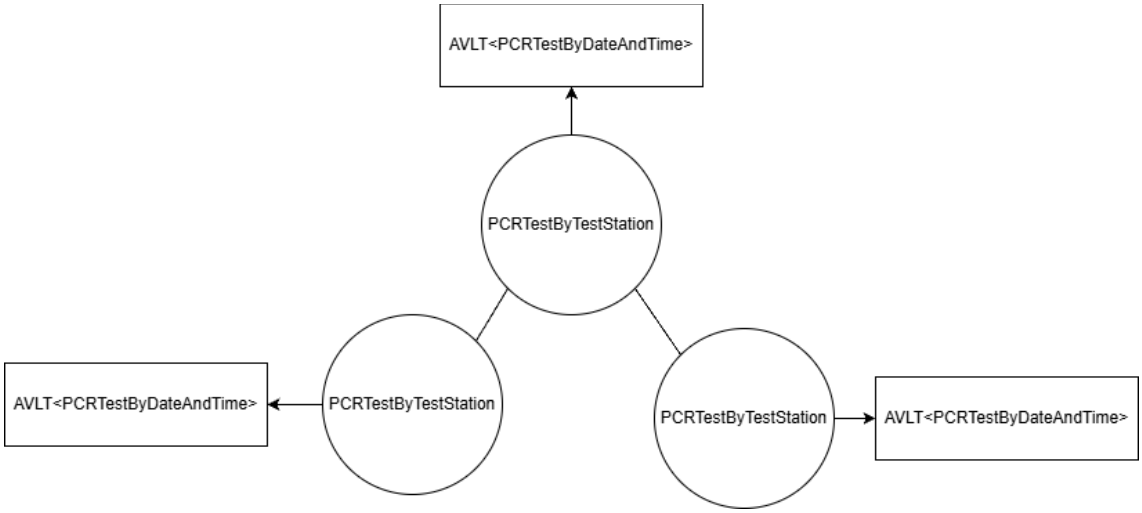
- **AVLT<PCRTestByDistrict> -** strom obsahuje všetky okresy v systéme usporiadané podľa čísla okresu. Každý okres následne obsahuje vlastné stromy AVLT<PCRTestByDateAndTime> (testy v okrese usporiadané podľa času vykonania) a AVLT<PCRTestByValueReverse> (testy v okrese usporiadané podľa hodnoty, ale zostupne).



- **AVLT<PCRTestByRegion> -** strom obsahuje všetky kraje v systéme usporiadané podľa čísla kraja. Každý kraj následne obsahuje strom AVLT<PCRTestByDateAndTime> (testy v kraji usporiadané podľa času vykonania).



- **AVLT<PCRTestByTestStation> -** strom obsahuje všetky odberné stanice v systéme usporiadané podľa čísla stanice. Každá stanica obsahuje strom AVLT<PCRTestByDateAndTime> (testy v stanici usporiadané podľa času vykonania).



3.5 Zložitosť operácií

1. **Vloženie výsledku PCR testu do systému.**
   - Zložitosť – **O(log (n)),** kde:
     - n – celkový počet PCR testov v systéme.
2. **Vyhľadanie výsledku testu (definovaný kódom PCR testu) pre pacienta (definovaný unikátnym číslom pacienta) a zobrazenie všetkých údajov.**
   - Zložitosť – **O(log (n) + log (k)),** kde:
     - n – celkový počet osôb v systéme,
     - k – počet testov priradených k danej osobe.
3. **Výpis všetkých uskutočnených PCR testov pre daného pacienta (definovaný unikátnym číslom pacienta) usporiadaných podľa dátumu a času ich vykonania.**
   - Zložitosť – **O(log (n) + k),** kde:
     - n – celkový počet osôb v systéme,
     - k – počet testov priradených k danej osobe.
4. **Výpis všetkých pozitívnych testov uskutočnených za zadané časové obdobie pre zadaný okres (definovaný kódom okresu).**
   - Zložitosť – **O(log (n) + k),** kde:
     - n – celkový počet unikátnych okresov v systéme,
     - k – počet testov v danom okrese v zadanom časovom intervale.

5. **Výpis všetkých testov uskutočnených za zadané časové obdobie pre zadaný okres (definovaný kódom okresu).**
    - Zložitosť – **O(log (n) + k),** kde:
        - n – celkový počet unikátnych okresov v systéme,
        - k – počet testov v danom okrese v zadanom časovom intervale.
6. **Výpis všetkých pozitívnych testov uskutočnených za zadané časové obdobie pre zadaný kraj (definovaný kódom kraja).**
    - Zložitosť – **O(log (n) + k),** kde:
        - n – celkový počet unikátnych krajov v systéme,
        - k – počet testov v danom kraji v zadanom časovom intervale.
7. **Výpis všetkých testov uskutočnených za zadané časové obdobie pre zadaný kraj (definovaný kódom kraja).**
    - Zložitosť – **O(log (n) + k),** kde:
        - n – celkový počet unikátnych krajov v systéme,
        - k – počet testov v danom kraji v zadanom časovom intervale.
8. **Výpis všetkých pozitívnych testov uskutočnených za zadané časové obdobie.**
    - Zložitosť – **O(log (n) + k),** kde:
        - n – celkový počet testov v systéme,
        - k – počet testov v zadanom časovom intervale.
9. **Výpis všetkých testov uskutočnených za zadané časové obdobie.**
    - Zložitosť – **O(log (n) + k),** kde:
        - n – celkový počet testov v systéme,
        - k – počet testov v zadanom časovom intervale.
10. **Výpis chorých osôb v okrese (definovaný kódom okresu) k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (X zadá užívateľ).**
    - Zložitosť – **O(log (n) + log (k) + m * log (h)),** kde:
        - n – celkový počet unikátnych okresov v systéme,
        - k – počet všetkých testov v danom okrese,
        - m – počet testov v danom okrese v časovom intervale,
        - h – počet pozitívnych osôb v danom okrese v danom časovom intervale.

11. **Výpis chorých osôb v okrese (definovaný kódom okresu) k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (X zadá užívateľ), pričom choré osoby sú usporiadané podľa hodnoty testu.**
    - Zložitosť – **O(log (n) + k * log (m)),** kde:
        - n – celkový počet unikátnych okresov v systéme,
        - k – počet všetkých testov v danom okrese,
        - m – počet pozitívnych osôb v danom okrese v časovom intervale.
12. **Výpis chorých osôb v kraji (definovaný kódom kraja) k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (X zadá užívateľ).**
    - Zložitosť – **O(log (n) + log (k) + m * log (h)),** kde:
        - n – celkový počet unikátnych krajov v systéme,
        - k – počet všetkých testov v danom kraji,
        - m – počet testov v danom kraji v časovom intervale,
        - h – počet pozitívnych osôb v danom kraji v danom časovom intervale.
13. **Výpis chorých osôb k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (X zadá užívateľ).**
    - Zložitosť – **O(log (n) + k * log (m)),** kde:
        - n – celkový počet testov v systéme,
        - k – počet testov v danom časovom intervale,
        - m – počet pozitívnych osôb v časovom intervale.
14. **Výpis jednej chorej osoby k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (X zadá užívateľ) z každého okresu, ktorá má najvyššiu hodnotu testu.**
    - Zložitosť – **O(n + k * log(m))**kde:
        - n – celkový počet testov v systéme,
        - k – počet unikátnych okresov v systéme,
        - m – počet osôb v systéme.

15. **Výpis okresov usporiadaných podľa počtu chorých osôb k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (X zadá užívateľ).**
    - Zložitosť – **O(n * (log (n) + log (k) + m * log (h))),** kde:
        - n – celkový počet unikátnych okresov v systéme,
        - k – počet všetkých testov v danom okrese,
        - m – počet testov v danom okrese v časovom intervale,
        - h – počet pozitívnych osôb v danom okrese v danom časovom intervale.
16. **Výpis krajov usporiadaných podľa počtu chorých osôb k zadanému dátumu, pričom osobu považujeme za chorú X dní od pozitívneho testu (X zadá užívateľ).**
    - Zložitosť – **O(n * (log (n) + log (k) + m * log (h))),** kde:
        - n – celkový počet unikátnych krajov v systéme,
        - k – počet všetkých testov v danom kraji,
        - m – počet testov v danom kraji v časovom intervale,
        - h – počet pozitívnych osôb v danom kraji v danom časovom intervale.
17. **Výpis všetkých testov uskutočnených za zadané časové obdobie na danom pracovisku (definované kódom pracoviska).**
    - Zložitosť – **O(log (n) + k),** kde:
        - n – celkový počet unikátnych staníc v systéme,
        - k – počet testov uskutočnených v danej stanici v zadanom časovom intervale.
18. **Vyhľadanie PCR testu podľa jeho kódu.**
    - Zložitosť – **O(log (n) + log(k)),** kde:
        - n – celkový počet testov v systéme,
        - k – celkový počet osôb v systéme.
19. **Vloženie osoby do systému.**
    - Zložitosť – **O(log (n)),** kde:
        - n – celkový počet osôb v systéme.
20. **Trvalé a nevratné vymazanie výsledku PCR testu (napr. po chybnom vložení), test je definovaných svojim kódom.**
    - Zložitosť – **O(log (n)),** kde:
        - n – celkový počet testov v systéme.

21. **Vymazanie osoby zo systému (definovaná rodným číslom unikátnym číslom pacienta) aj s jej výsledkami PCR testov.**
    - Zložitosť – **O(log (n) + k * log(m)),** kde:
        - n – celkový počet osôb v systéme,
        - k – celkový počet testov danej osoby,
        - m – celkový počet testov v systéme.

4 DIAGRAM TRIED

Diagram tried je dostupný aj ako .png súbor v priečinku s dokumentáciou.

